UNIVERSIDAD NACIONAL DE JULIACA

ESCUELA PROFESIONAL DE INGENIERÍA DE SOFTWARE Y SISTEMAS

Práctica N.° 2: Programación orientada a objetos en Kotlin

Curso: Programación de Equipos Móviles

Proyecto: ProgramacionEquiposMoviles

Rama evaluada: `practica-02`

Integrantes:

- Juan Gedeon Tito Moya
- Yuri Andrea Ramos Suca

Año 2026

1. OBJETIVO

Aplicar los fundamentos de programación orientada a objetos y las características avanzadas de Kotlin dentro de una aplicación Android, comprobando su correcta compilación y comportamiento mediante pruebas unitarias automatizadas.

2. DESARROLLO

La práctica se implementó en la rama `practica-02`, commit `d06be38`. La función de nivel superior `main()` se invoca desde `onCreate()` de `MainActivity`, y la etiqueta `PRACTICA-02` permite identificar en Logcat los mensajes generados por los ejercicios.



Contenidos implementados

Clases y objetos

Singleton, companion object, objetos anónimos, atributos, métodos y distintos constructores.

Funciones Kotlin

Funciones de nivel superior, extensiones, parámetros predeterminados y `vararg`.

Diseño orientado a objetos

Encapsulamiento, herencia, polimorfismo, abstracción e interfaces.

Características del lenguaje

Lambdas, clases de datos, destructuración, propiedades calculadas y operadores.

Estructura del informe

Pruebas de compilación y ejecución	**3**
Pruebas funcionales de los contenidos	**4–9**
Pruebas unitarias automatizadas	**10**
Resultado y conclusiones	**11**

VERIFICACIÓN DEL PROYECTO

3. PRUEBAS DE COMPILACIÓN Y EJECUCIÓN

3.1 Compilación del proyecto Android

Se compiló la variante de depuración mediante Gradle. El proceso completó las tareas de recursos, manifiesto, compilación Kotlin, generación DEX y empaquetado del APK.

Terminal · rama practica-02

```
$ ./gradlew assembleDebug --console=plain
> Task :app:compileDebugKotlin UP-TO-DATE
> Task :app:dexBuilderDebug
> Task :app:packageDebug
> Task :app:assembleDebug

BUILD SUCCESSFUL in 9s
36 actionable tasks: 3 executed, 33 up-to-date
```

Resultado: compilación satisfactoria. Se generó `app-debug.apk` con un tamaño de 11 789 866 bytes (aprox. 12 MB).

3.2 Punto de entrada de la práctica

MainActivity.kt — inicio de ejecución

```kotlin
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        main()
        enableEdgeToEdge()
        setContent { /* interfaz Compose */ }
    }
}

object configurationApp {
    const val TAG = "PRACTICA-02"
    const val VERSION = "1.0.2"
    var userActive: String = "guest"
}
```

La llamada directa a `main()` hace que los casos de demostración se ejecuten al iniciar la actividad. La constante `TAG` centraliza el filtro utilizado por los mensajes de diagnóstico.

85	604	0
archivos Kotlin de producción	líneas en MainActivity.kt	errores de compilación

3.3 OBJETOS, CLASES Y ENCAPSULAMIENTO

Se probaron tres modalidades de objetos: un singleton declarado con `object`, un `companion object` accesible desde su clase y un objeto anónimo que implementa una interfaz.

MainActivity.kt — objetos

```
object configurationApp {
    const val attempts = 5
    fun mostrarInfo() = Log.d(TAG,
        "My version: $VERSION, Usuario: $userActive")
}

class validator {
    companion object {
        const val MIN_AGE = 18
        fun isvalid(age: Int) = age >= MIN_AGE
    }
}

val myNotification = object : notification {
    override fun send() {
        Log.d(TAG, "Send notification anonymous ...")
    }
}
```

Validación y protección del estado

Las clases `Temperature`, `Counter`, `UserE`, `SavingsAccount` y `ProductEC` aplican encapsulamiento para controlar el acceso y mantener valores válidos.

Temperature.kt — propiedad validada

```
var celsius: Double = 0.0
    set(value) {
        if (value >= -273.15) field = value
        else Log.d(TAG, "Invalid temperature")
    }

val fahrenheit: Double
    get() = celsius * 9 / 5 + 32
```

> **Comprobación:** al asignar `100.0` se obtiene `212.0 °F`; el intento de asignar `-300.0` es rechazado por estar por debajo del cero absoluto.

Clase	Mecanismo aplicado	Finalidad
Counter	`private set`	Modificar el contador solo mediante métodos.
UserE	Contraseña privada	Exponer únicamente una operación de verificación.
SavingsAccount	Saldo privado	Validar depósitos y retiros.
ProductEC	Constructor privado y fábrica	Crear productos con precio válido.

3.4 FUNCIONES Y EXTENSIONES

La práctica incluye funciones de nivel superior con expresiones compactas, argumentos predeterminados y reglas de negocio, además de extensiones sobre tipos existentes.

MainActivity.kt — funciones de nivel superior

```kotlin
fun calculateIGV(amount: Double, percentage: Double = 0.18) =
    amount * percentage

fun obtainCategory(score: Int): String = when {
    score >= 18 -> "Excellent"
    score >= 14 -> "Well"
    score >= 11 -> "Regular"
    else        -> "Disapproved"
}
```

Funciones de extensión

MainActivity.kt / ProductF.kt — extensiones

```kotlin
fun String.capitalizar() =
    trim().replaceFirstChar { it.uppercase() }

fun Int.factorial(): Long =
    if (this <= 1) 1 else this * (this - 1).factorial()

fun ProductF.priceWithIGV() = price * 1.18
fun ProductF.isAvailable() = stock > 0
fun List<ProductF>.inventoryTotal() =
    sumOf { it.price * it.stock }
```

Parámetros variables

`sumAll(vararg)`

Recibe una cantidad variable de números y devuelve su suma.

`showOptions(vararg)`

Enumera opciones con `forEachIndexed`.

`greatest(vararg)`

Obtiene el mayor entero o devuelve cero.

Visibilidad

Se probaron funciones públicas, `internal` y `private`.

Expresión ejecutada	Resultado esperado/verificado
`calculateIGV(250.0)`	`45.0`
`" hello world".capitalizar()`	`"Hello world"`
`5.factorial()`	`120`
`sumAll(1.0, 2.5, 3.0, 4.5)`	`11.0`

3.5 CONSTRUCTORES, RECURSOS Y SOBRECARGA

Constructores

Se implementaron constructores primarios, bloques `init`, constructores secundarios, argumentos predeterminados y argumentos con nombre. Las clases `PersonCP`, `PersonCBI`, `PersonCS`, `PersonSVD`, `ProductAN`, `AccountSC` y `VehicleBI` muestran estas variantes.

MainActivity.kt — sobrecarga de constructores

```
PersonSVD("Jean", 25, "jean@mail.com").logData()
PersonSVD("Ana", 30).logData()
PersonSVD("Luis").logData()

ProductAN(name = "Laptop", price = 3500.0).logInfo()
ProductAN(name = "Mouse", stock = 50,
          category = "Peripherals").logInfo()
```

Gestión determinista de recursos

Las clases `DatabaseConnection` y `UserSession` implementan `Closeable`. El bloque `use` garantiza el cierre del recurso incluso si ocurre una excepción.

MainActivity.kt — recursos

```
DatabaseConnection("users.db").use { database ->
    Log.d(TAG, database.query("SELECT * FROM users"))
}

UserSession("user_42").use {
    Log.d(TAG, it.obtainData())
}
```

Sobrecarga de métodos y operadores

`Calculator` sobrecarga `add` para enteros, decimales, tres valores y listas. `Vector` redefine `plus`, `minus` y `times`; `Matrix` redefine `get` y `set`.

MainActivity.kt — operadores

```
val vector1 = Vector(3.0, 4.0)
val vector2 = Vector(1.0, 2.0)
Log.d(TAG, "${vector1 + vector2}")   // Vector(4.0, 6.0)
Log.d(TAG, "${vector1 * 2.0}")       // Vector(6.0, 8.0)

val matrix = Matrix(2, 3)
matrix[1, 2] = 5.0
Log.d(TAG, "${matrix[1, 2]}")
```

REUTILIZACIÓN Y COMPORTAMIENTO DINÁMICO

3.6 HERENCIA Y POLIMORFISMO

Las jerarquías de animales, figuras, vehículos, productos, notificaciones y cuentas muestran la reutilización de estado y comportamiento mediante clases `open`, métodos sobrescritos y llamadas a `super`.



MainActivity.kt — colección polimórfica

```
val animalsP: List<AnimalP> = listOf(
    DogP("Rex"), CatP("Luna"), CowP("Lola"),
    DogP("Max"), CatP("Mishi")
)
processAnimals(animalsP)
```

Polimorfismo con figuras

MainActivity.kt / ShapeP.kt — área polimórfica

```
val shapesP: List<ShapeP> = listOf(
    RectangleP("red", 5.0, 3.0),
    CircleP("blue", 4.0),
    TriangleP("green", 6.0, 4.0)
)
shapesP.forEach { showShapeInfo(it) }
Log.d(TAG, "Total area: ${calculateTotalArea(shapesP)}")
```

La función `calculateTotalArea` opera sobre el tipo base `ShapeP`; cada subclase aporta su propia implementación de `calculateArea()`. También se probaron comprobación y conversión segura de tipos con `is` y `as?`.

Jerarquía	Subclases utilizadas	Comportamiento especializado
AnimalH	DogH, CatH	Sonido, búsqueda y escalada.
ShapeH / ShapeP	Rectángulo, círculo, triángulo, pentágono, rombo	Cálculo de áreas.
VehicleH	SportsCar, PickupTruck	Turbo y capacidad de carga.
ProductH	DigitalProduct, PhysicalProduct	Descarga y costo de envío.
NotificationH	UrgentNotificationH, SilentNotificationH	Registro y presentación diferenciada.

3.7 ABSTRACCIÓN, INTERFACES Y COLABORACIÓN

Clases abstractas

Las clases `EmployeeA`, `PaymentProcess`, `GameA` y `NotificationA` definen una base común y delegan a las subclases la implementación de operaciones específicas.

MainActivity.kt — abstracción

```
val team: List<EmployeeA> = listOf(
    Developer("Jean", "EMP-001", "Kotlin"),
    ProjectManager("Ana", "EMP-002", 5),
    Designer("Luis", "EMP-003")
)
team.forEach { it.showInfo() }
```

Interfaces

Se implementaron contratos para exportación, persistencia, documentos, registro del sistema y notificaciones. La interfaz permite cambiar la implementación sin modificar el código consumidor.

Contrato	Implementaciones	Prueba realizada
Exportable	PdfExporter, CsvExporter, JsonExporter	Validar y exportar la misma cadena.
Registrable	UserRepository, ProductRepository	Registrar, buscar y eliminar.
NotifierI	Email, SMS, Push	Enviar la misma alerta por tres canales.
Printable / Shareable / Downloadable	DocumentI, ImageI	Combinar múltiples capacidades.

Colaboración y composición

CarC + Engine

El automóvil delega al motor las operaciones de encendido y apagado.

PrintOffice + Printer

La oficina coordina una impresora y un documento.

Classroom + StudentC

Agrega estudiantes, muestra resultados y calcula el promedio.

SmartWatch + Battery + Gps

Compone capacidades independientes en un dispositivo.

MainActivity.kt — inyección por interfaz

```
OrderService(OrderEmail("jean@mail.com"))
    .confirmOrder("ORD-001")

OrderService(OrderSms("+51 999 888 777"))
    .cancelOrder("ORD-002")
```

3.8 LAMBDAS, OBJETOS ANÓNIMOS Y DATA CLASS

Lambdas y funciones de orden superior

Las lambdas se almacenan en variables, se asignan como eventos y se reciben como parámetros. Esto permite separar la operación de la lógica que decide cómo transformar, validar, filtrar o presentar datos.

MainActivity.kt — lambdas

```
val greetingLambda: (String) -> String =
    { name -> "Hello, $name!" }
val duplicateLambda: (Int) -> Int = { it * 2 }

val buttonL = ButtonL("Send")
buttonL.onClick = {
    Log.d(TAG, "Sending form...")
    Log.d(TAG, "Validating data...")
}
buttonL.onValidate = { text -> text.length >= 3 }
```

También se utilizaron comparadores y listeners creados como objetos anónimos, además de una `fun interface` instanciada mediante SAM conversion.

Clases de datos

MainActivity.kt — igualdad, copia y destructuración

```
val pointData1 = PointData(3, 5)
val pointData2 = PointData(3, 5)
Log.d(TAG, "${pointData1 == pointData2}")  // true

val configBase = ConfigurationData()
Log.d(TAG, "${configBase.copy(theme = "dark")}")

val coordinate = CoordinateData(-16.4, -71.5, 2335.0)
val (latitude, longitude, altitude) = coordinate
```

Característica	Uso comprobado
`equals()` por valor	Dos `PointData(3, 5)` son iguales, a diferencia de dos objetos normales.
`copy()`	Se crea una configuración modificando solo tema, idioma o tamaño de fuente.
Destructuración	Se extraen coordenadas y el resultado de una búsqueda en variables separadas.
Métodos propios	`EmployeeData` calcula bonificación y determina antigüedad.

Resultado: se verificó que las clases de datos reducen código repetitivo y representan correctamente valores comparables, copiables y destructurables.

EVIDENCIA AUTOMATIZADA

3.9 PRUEBAS UNITARIAS

Se ejecutó la tarea `testDebugUnitTest`. El proyecto contiene seis casos específicos para los ejercicios del capítulo y un caso base del proyecto; los siete finalizaron correctamente.

Terminal · Gradle Test Runner

```
$ ./gradlew testDebugUnitTest --console=plain
> Task :app:compileDebugUnitTestKotlin UP-TO-DATE
> Task :app:testDebugUnitTest UP-TO-DATE

BUILD SUCCESSFUL in 10s
24 actionable tasks: 4 executed, 20 up-to-date
```

7	0	0
pruebas ejecutadas	fallos	errores

Casos verificados

Prueba	Aspecto cubierto	Estado
topLevelAndExtensionFunctionsReturnExpectedValues	IGV, capitalización, correo y factorial.	APROBADA
productExtensionsCalculateAvailabilityAndTax	Disponibilidad y precio con IGV.	APROBADA
overloadedOperatorsProduceExpectedResults	Suma, multiplicación y magnitud de vectores.	APROBADA
matrixIndexOperatorsStoreAndReadValues	Operadores indexados `get`/`set`.	APROBADA
calculatedPropertiesAndPolymorphismReturnExpectedAreas	Propiedades calculadas y área total.	APROBADA
dataClassSupportsValueEqualityCopyAndDestructuring	Igualdad, copia y destructuración.	APROBADA
addition_isCorrect	Prueba base del proyecto.	APROBADA

Fuente de evidencia: archivos XML de resultados generados en `app/build/test-results/testDebugUnitTest/`. Los registros reportan `tests="7"`, `failures="0"` y `errors="0"`.

SÍNTESIS

4. RESULTADO DE LA PRÁCTICA

La rama `practica-02` compila correctamente, genera un APK de depuración y supera todas las pruebas unitarias disponibles. La implementación demuestra el uso integrado de los principales mecanismos de programación orientada a objetos e idiomáticos de Kotlin.

Resultados verificados

- Creación de objetos singleton, companion objects y objetos anónimos.
- Modelado mediante clases, propiedades, métodos y diferentes tipos de constructores.
- Encapsulamiento del estado con validación, getters, setters y `field`.
- Funciones de nivel superior, extensiones, argumentos predeterminados y `vararg`.
- Sobrecarga de métodos y operadores para vectores, matrices y comparaciones.
- Herencia, sobrescritura, polimorfismo, clases abstractas e interfaces.
- Composición y colaboración entre objetos con responsabilidades separadas.
- Lambdas, funciones de orden superior, listeners y clases de datos.
- Gestión segura de recursos mediante `Closeable` y `use`.

Conclusiones

1. Organización del código

La separación en clases pequeñas permite representar responsabilidades concretas y facilita la reutilización de comportamiento.

2. Seguridad del estado

El encapsulamiento y la validación de propiedades evitan valores inválidos y reducen cambios accidentales desde otras clases.

3. Flexibilidad

El polimorfismo, las interfaces y la composición permiten sustituir implementaciones sin modificar a sus consumidores.

4. Confiabilidad

Las pruebas automáticas confirmaron los resultados de funciones, operadores, propiedades calculadas, polimorfismo y clases de datos.

Resumen de evidencia

Elemento	Resultado
Rama / commit	`practica-02` / `d06be38`
Compilación	**BUILD SUCCESSFUL**
APK generado	`app-debug.apk` · 11 789 866 bytes
Pruebas unitarias	**7 aprobadas · 0 fallos · 0 errores**
Fecha de verificación	21 de septiembre de 2026